
Machine Learning Fights Back

Alexandra Von Braunschweig¹ Jade Carpentier¹ Thomas Cherqui¹ Hannah O’Keeffe¹

Abstract

This work challenges the claim that established machine learning methods cannot achieve high performance on the Kryptonite-2.0 dataset (Quinn & Luther, 2025). We evaluate two distinct methodologies, an instance based method (k-nearest neighbours) and a parametric model (a multi-layer perceptron) to assess their ability to learn from the dataset’s irregular and highly non-linear structure. We obtain high accuracies, substantially improving the “as-good-as-random” baseline. These findings suggest that the reported failure of machine learning methods to learn Kryptonite-2.0 does not arise from limitations of universal function approximation, but rather from specific methodological choices applied to this particularly complex dataset.

1. Introduction

As machine learning methods have become increasingly sophisticated, they have demonstrated the capacity to solve progressively more complex problems. This progress naturally raises the question of whether there exist datasets fundamentally too challenging for contemporary approaches. The development of challenge datasets represents an increased demand for robust benchmarks that can highlight limitations of existing methods and direct future research. The Kryptonite-2.0 dataset is a notable example that seeks to highlight the faults of modern methods through what might be misjudged as a simple binary classification task.

This paper seeks to address the problem by first investigating the dataset to understand where its challenges lie. We analyse the faulty foundations of prior work to identify why these approaches were unable to achieve high accuracy. Building on this analysis, we experiment with different

¹Department of Computing, Imperial College London, London, United Kingdom. Correspondence to: Alexandra von Braunschweig <amv25@imperial.ac.uk>, Jade Carpentier <jc2425@ic.ac.uk>, Thomas Cherqui <tac25@ic.ac.uk>, Hannah O’Keeffe <heo25@ic.ac.uk>.

modelling approaches that are better able to handle the irregular patterns in the data. By systematically evaluating our approaches, we demonstrate that each model improves significantly on prior attempts and challenge the claim that Kryptonite-2.0 cannot be effectively modelled.

To this end, we evaluate two methods that impose few assumptions about the feature space. The first, k-nearest neighbours (kNN), is a non-parametric approach that does not learn explicit feature representations but instead naturally follows irregular decision boundaries. The second, a multi-layer perceptron (MLP), learns feature representations through non-linear activation functions and regularisation to tackle the irregularities. We carry out a series of experiments on both models and assess performance relative to the thresholds laid out in Quinn & Luther (2025). To ensure reliability, we report accuracy estimates averaged over multiple seeds and quantify statistical uncertainty by reporting the confidence on our accuracy estimates.

Both models achieve accuracies above 90% on $n = 10$ and $n = 12$ feature sets. While the performance of kNN drops as dimensionality increases, the MLP achieves an accuracy of 79% on the full $n = 20$ feature space. The results show that Kryptonite-2.0 is not unsolvable by modern machine learning methods and highlight the importance of carefully considering methodological choices and imposed feature structure in every machine learning problem.

2. Methodology

2.1. Discussion of the foundations of the Kryptonite-2.0 paper

The Kryptonite-2.0 paper sets out to question the assumption that Machine Learning models can approximate any function, aiming to challenge the Universal Approximation Theorem (UAT).

The UAT provides a mathematical justification for the use of neural networks. It states that a feedforward neural network with a single hidden layer and a non-polynomial activation function σ (such as ReLU or the sigmoid function) can approximate any continuous function f on a compact domain $D \subset \mathbb{R}^n$ arbitrarily well.

Formally, it means that for any $\varepsilon > 0$, there exists a neural

network function $\hat{f}$ of the form

$$\hat{f}(x) = \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i)$$

such that

$$\sup_{x \in D} |f(x) - \hat{f}(x)| < \varepsilon$$

where m is the number of hidden neurons, and a_i, w_i, b_i are the network parameters.

This is an existence theorem – it guarantees that some network architecture exists to approximate the target function, but it does not guarantee easy construction or training of such a network. Thus, the paper does not refute the theory but questions its practical limitations, such as insufficient and noisy data or inherent difficulty of optimising in a complex loss landscape.

While this highlights the theoretical misinterpretation, the experimental design itself also suffers from rigid modelling assumptions. The models described in the Kryptonite-2.0 paper lack flexibility due to fixed design choices such as a predetermined basis expansion and a constant regularisation parameter. In our work, we address these limitations by introducing learnable and stochastic components, making the experimental setup better aligned with the data.

2.1.1. BASIS EXPANSION FOR NON-LINEARITY

A key technique for enabling linear models to capture non-linear relationships is Basis Expansion. This involves a function $\phi(x)$ that maps the original feature vector x into a new, often higher-dimensional, feature space. The link function g^{-1} then relates the model’s linear predictor, $\theta^\top \phi(x)$, to the final output y .

$$\hat{y} = g^{-1}(\theta^\top \phi(x))$$

The Kryptonite-2.0 paper proposes Polynomial Basis Expansion, as well as a GPT-based basis expansion, where a pre-trained GPT-2 model was used to convert input vectors into high-dimensional text embeddings.

While the polynomial approach is constrained by the choice of a fixed basis, the GPT-based approach replaces the basis with semantic embeddings that might not be aligned with the underlying data distribution or task objectives. Applying a GPT model to a classification task of this form appears inappropriate, as the model is not optimised for detecting structured patterns in tabular data (G. Badaro, 2023) and the Kryptonite 2.0 dataset is deliberately designed to obscure patterns and hinder straightforward feature correlations. As a result, this approach introduces unnecessary complexity

without providing a meaningful inductive bias for the problem at hand. Instead, we propose using a Multi-Layer Perceptron as a parameterised basis expansion that is learned from the data.

2.2. The Binary Classification Problem

Binary classification is a supervised learning task where the goal is to learn a model $f_\theta : R^n \rightarrow \{0, 1\}$ that predicts the label of a data point.

Given a labelled dataset $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, with $x^{(i)} \in R^n$ and $y^{(i)} \in \{0, 1\}$, the model parameter θ is learned by minimizing a loss function that measures prediction error over D .

2.3. Nearest Neighbours

k-NN is one of the most intuitive and widely used methods for classification in Machine Learning. The underlying idea is straightforward: the label assigned to a given data point x is determined by the majority label among its k closest samples in the training set. These samples are selected according to a chosen distance metric such as the Euclidean distance.

Mathematically, let $\mathcal{N}_k(x)$ denote the set of indices of the k nearest neighbours of x . The predicted label $\hat{y}(x)$ is then given by:

$$\hat{y}(x) = \arg \max_{c \in \mathcal{Y}} \sum_{i \in \mathcal{N}_k(x)} 1_{[y_i=c]}$$

where $\mathcal{Y}$ is the set of possible classes and $1_{[y_i=c]}$ is an indicator function that equals 1 if $y_i = c$, and 0 otherwise.

2.4. Multilayer Perceptron for Binary Classification

2.4.1. MODEL ARCHITECTURE

A Multilayer Perceptron (MLP) is a feedforward neural network capable of approximating complex, non-linear functions. For binary classification, the MLP estimates the probability that a given input $x \in R^d$ belongs to one of two classes, denoted $y = 0$ or $y = 1$. The network computes this probability as a composition of linear transformations and non-linear activations across multiple layers:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}), \quad l = 1, \dots, L, \quad (1)$$

where $h^{(0)} = x$, $W^{(l)}$ and $b^{(l)}$ are the weights and biases of the l -th layer, $\sigma(\cdot)$ denotes the activation function (here ReLU: $\sigma(z) = \max(0, z)$), and L is the number of hidden layers. The final layer produces a single logit $z^{(\text{output})}$, which is converted to a probability using the sigmoid function:

$$P(y = 1 | x; \theta) = \frac{1}{1 + \exp(-z^{(\text{output})})}, \quad (2)$$

where θ represents all trainable parameters (weights and biases) of the network.

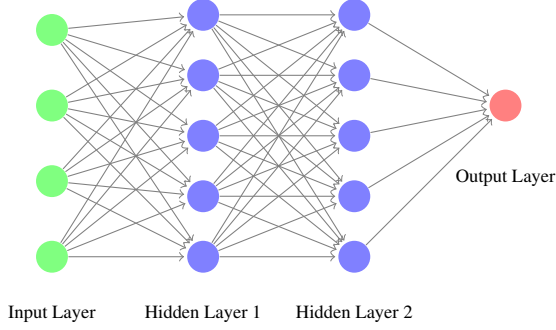


Figure 1. Architecture of the binary classification MLP with two hidden layers.

2.4.2. LOSS FUNCTION

The network is trained by minimizing the binary cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)], \quad (3)$$

where $p_i = P(y_i = 1 | x_i; \theta)$ is the predicted probability and $y_i \in \{0, 1\}$ is the true label.

2.4.3. REGULARISATION TECHNIQUES

To prevent overfitting and enhance generalisation, we apply three regularisation techniques in the MLP, unlike the Kryptonite-2.0 paper, which relied solely on a fixed weight-decay regularisation.

Dropout randomly deactivates a fraction p of neurons in each hidden layer during training:

$$\tilde{h}^{(l)} = r^{(l)} \odot h^{(l)}, \quad r^{(l)} \sim \text{Bernoulli}(1 - p), \quad (4)$$

where $h^{(l)}$ represents the activations of the l -th hidden layer, $\odot$ denotes element-wise multiplication, and $r^{(l)}$ is a random binary mask. Dropout forces the network to rely on multiple independent pathways, reducing overfitting.

Batch normalization normalizes the activations of each hidden layer to have zero mean and unit variance within a mini-batch:

$$\hat{h}^{(l)} = \frac{h^{(l)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad h_{\text{BN}}^{(l)} = \gamma \hat{h}^{(l)} + \beta, \quad (5)$$

where μ_B and σ_B^2 are the mean and variance of the mini-batch, ϵ is a small constant for numerical stability, and γ, β are learnable scale and shift parameters. Batch normalization stabilizes training and has a mild regularisation effect.

Weight decay further mitigates overfitting by penalizing large parameter values in the network, leading to smoother decision boundaries and improved generalisation. The loss function is then:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}(\theta) + \lambda \|\theta\|_2^2, \quad (6)$$

where λ controls the strength of the penalty.

These techniques, combined with careful selection of hyperparameters, help the MLP generalise better to unseen data.

2.4.4. LEARNING ALGORITHM

Since the optimisation problem has no closed-form solution, we use gradient-based methods. In particular, parameters are updated iteratively using stochastic gradient descent:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}(\theta; X_b), \quad (7)$$

where η is the learning rate, $\nabla_{\theta} \mathcal{L}(\theta; X_b)$ is the gradient of the loss computed over a mini-batch $X_b \subseteq X$, and t indexes the iteration. The process is repeated for a fixed number of epochs.

3. Experimental Design

3.1. Datasets

We evaluate the models on datasets with similar feature distributions but varying input dimensionality to test generalisation and robustness. LDA (Linear Discriminant Analysis) on the 10-dimensional feature space shows substantial class overlap, indicating no linear decision boundary. Pairwise feature plots confirm weak separability, suggesting that class discrimination relies on higher-dimensional feature interactions. PCA (Principal Component Analysis) further reveals evenly distributed variance and weak linear correlations, supporting the need for a non-linear model like an MLP (see Section A).

For each dataset, we use a standard train / validation / test split (70%, 15%, 15%) to ensure unbiased evaluation.

3.2. Hypothesis and Target Performance

To refute the claim that Kryptonite-2.0 is unsolvable, we hypothesize that a sufficiently chosen model can reach classification accuracy above the thresholds set by the original authors. Therefore, we consider the dataset to be solvable at dimension n if a model achieves accuracy greater than the reported target for that value of n (Table 1).

Number of features	10	12	14	16	18	20
Target Accuracy (%)	94	93	92	91	80	75

Table 1. Accuracy targets for each value of n in our datasets.

3.3. K-Nearest Neighbours Setup

We use 5-fold cross-validation on the training and validation data to select the optimal number of neighbours based on validation accuracy. Then, we test the model with the best hyper parameter k on the test-dataset.

3.4. MLP Setup

3.4.1. EXPERIMENTAL SETUP

The performance of the MLP depends on several key hyperparameters, including the learning rate η which controls the step size in gradient descent and the batch size which sets the number of samples used to compute each gradient update.

We perform a grid search over these parameters to identify stable training configurations. Each combination is evaluated across three random seeds to ensure robustness to initialization and to ensure our results are not due to randomness in the seed. Tested parameters are summarized in Table 2.

We set some hyperparameters, such as weight decay, to their default values to avoid substantially expanding the grid search and increasing the computation time, which creates issues discussed in Section 5.2

Hyperparameter	Value
Learning rate (η)	$1 \times 10^{-3}, 3 \times 10^{-3}, 1 \times 10^{-2}$
Batch size	64, 256
Random seed	1, 2, 3
Weight decay	10^{-4}
Number of epochs	80

Table 2. Hyperparameters tested across all datasets.

3.4.2. HYPERPARAMETER TUNING PROCEDURE

We evaluate each configuration using grid search, monitoring both training and validation losses across different combinations of learning rate and batch size.

We implement early stopping for $n = 10$ and $n = 12$ meaning training is halted if the loss does not show a significant improvement for 10 consecutive epochs.

3.4.3. CONVERGENCE ANALYSIS

Figure 2 illustrates the impact of these hyperparameters on convergence for $n = 10$, where each curve represents the mean training loss over three random seeds and the shaded regions indicate one standard deviation.

Although neural network optimisation lacks theoretical guarantees of convergence due to its non-convex landscape (Goodfellow et al., 2016), clear empirical differences

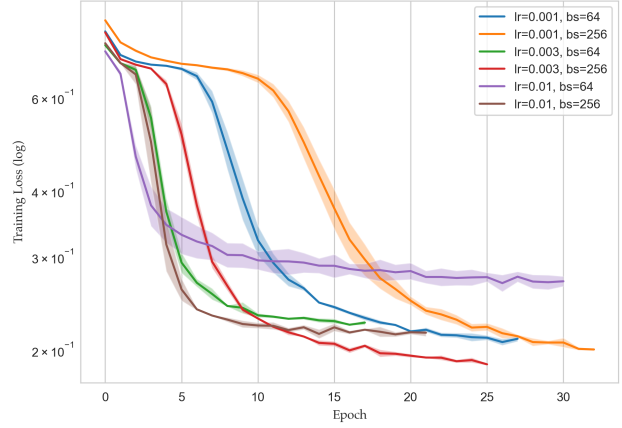


Figure 2. Loss convergence for different hyper-parameters averaged across seeds for $n = 10$

emerge across configurations. A learning rate of $\eta = 10^{-3}$ with a batch size of 64 provides the most stable convergence, for $n = 10$. In fact, repeating this experiment for every n , results in the same chosen optimal hyperparameters as shown in Section 4.2. Thus, for every dataset the best hyperparameters configuration used for testing is a learning rate of $\eta = 10^{-3}$ with a batch size of 64.

For $n = 10$, most configurations plateau near epoch 15, justifying the use of early stopping to prevent overfitting (Goodfellow et al., 2016). In higher-dimensional datasets, this plateau occurs later, reflecting increased optimisation complexity.

3.5. Statistical Validity

3.5.1. WEAK LAW OF LARGE NUMBERS

To calculate the confidence that the measured accuracy $\hat{r}_k$ is a close approximation of the model's true, generalised accuracy μ , we apply the following bound derived from the Weak Law of Large Numbers:

$$P(|\mu - \hat{r}_k| \geq \epsilon) \leq \frac{\sigma^2}{k\epsilon^2} \leq \frac{0.25}{k\epsilon^2}$$

As the accuracy r_i of a single trial in a binary classification task can be either 0 or 1, the random variable is bounded between 0 and 1. Therefore, we know that $\sigma^2 \leq 0.25$.

The initial calculation using the Weak Law of Large Numbers with a $k = 3,000$ samples in the test set and $\epsilon = 0.01$ showed a low 17% certainty that the measured accuracy is within $\pm 1\%$ of the true accuracy. However, this is sufficient, as the paper aims only to demonstrate that the model beats the target accuracy. By setting the error margin ϵ as the difference between the observed (Table 5) and target accuracy, we can demonstrate a high confidence in the model beating

the target threshold (Table 3).

# of features	10	12	14	16	18	20
Target Accuracy (%)	94	93	92	91	80	75
Actual Accuracy (%)	96	96	96	93	86	79
Error margin ϵ (%)	2	3	4	2	6	4
Confidence (%)	79	91	95	79	98	95

Table 3. Lower bound of the confidence of beating the target accuracy for each of the datasets.

3.5.2. GENERALISATION BOUND FOR MODEL SELECTION

While the Weak Law of Large Numbers provides a confidence bound for a single model, our experimental design requires us to select the best model from a set Θ . We must therefore account for the risk that one of these models looks good purely by chance on our validation data, while another model would actually fit the data better.

We can bound this generalisation error by considering the probability that the worst model in our set has a large gap between its true risk (R) and its empirical risk (R_{emp}), effectively bounding the probability that any model in our set Θ is misleading. The generalisation bound is therefore given by the following :

$$P\left[\left\|R(\theta) - R_{emp}(\theta)\right\|_{\infty} > \epsilon\right] \leq \sum_{\theta \in \Theta} P\left[|R(\theta) - R_{emp}(\theta)| > \epsilon\right] \leq \frac{|\Theta|}{4n\epsilon^2}.$$

In our experiments, we tested $|\Theta| = 6$ different parameter settings, and our validation set had 3,000 samples for each of the 6 datasets. Since the chosen model consistently performed best on each of these datasets with their distinct number of features, we had a total of $n = 18,000$ validation set samples. When choosing $\epsilon = 0.02$, the risk of selecting a bad model is upper bounded by 0.21. This could be improved by adding more validation data.

4. Experimental Results

4.1. k-NN

After computing k-NN and finding the optimal k through cross-validation, we obtain the test accuracies presented in Table 4.

We observe a sharp drop in accuracy as the number of features increases. The accuracy of our model exceeds the threshold for $n = 10$, but performance declines to below 50% for $n = 18$ and $n = 20$ (comparable to a random classifier).

# of features	10	12	14	16	18	20
Optimal k	5	1	1	1	1	1
Test Accuracy (%)	96	92	82	59	47	49

Table 4. Test accuracy for each number of features n in the dataset.

k-NN therefore highlights the impact of the feature space dimensionality on model performance — this is the well-known *curse of dimensionality* (Chattopadhyay et al., 2019). As the number of features increases, the Euclidean distance between points becomes less meaningful, and local neighbourhoods lose discriminative power. This explains why, from $n = 12$ onwards, the optimal number of neighbours chosen is $k = 1$.

4.2. MLP Results

Our MLP results support the hypothesis that the model meets or exceeds the target accuracy for all n . Figure 3 shows validation accuracy curves across feature dimensions, averaged over 3 seeds (shaded regions indicate one standard deviation). Lower-dimensional datasets converge rapidly within 20–30 epochs as referenced in Section 3.4.3, while higher-dimensional datasets converge more slowly with slightly lower accuracy. Nevertheless, even for $n = 20$, the model shows clear learning progress and stabilizes above the performance threshold.

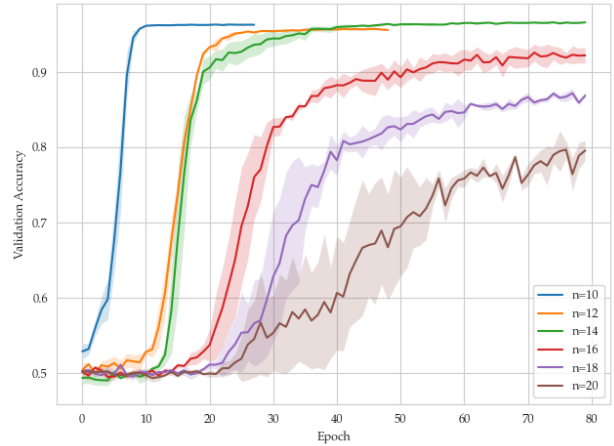


Figure 3. Validation accuracy curves for each n showing the mean accuracy (solid line) and standard deviation (shaded region) across seeds using $\eta = 10^{-3}$ and batch size = 64.

After selecting the best-performing hyperparameters using the validation set, we evaluate the final models on the held-out test set. Table 5 reports the mean test accuracy and standard deviation across seeds. In all cases, the test performance exceeds the required threshold, indicating that the

models generalise reliably beyond the training and validation data.

# of Features	10	12	14	16	18	20
Test Acc. (%)	96.3	95.8	96.4	93.1	85.8	78.9
Std. Dev. ($\cdot 10^{-2}$)	0.0	0.1	0.0	1.0	0.4	0.6

Table 5. Test accuracies using $\eta = 10^{-3}$ and batch size = 64.

5. Discussion

5.1. Key findings

This work demonstrates that the Kryptonite-2.0 dataset, while complex and highly non-linear, is not inherently unsolvable by standard machine learning methods. Through exploratory analysis using PCA and LDA (Appendix A), we confirmed the absence of linear separability, highlighting the dataset’s challenging structure. Our experiments show that k-nearest neighbours can capture low-dimensional patterns but its performance declines as dimensionality increases. In contrast, a well-tuned multilayer neural network consistently achieves high accuracy even on the full 20-dimensional feature set.

These findings suggest that prior reported failures are not due to fundamental limitations of machine learning models, but rather result from methodological choices, such as inadequate model flexibility or insufficient hyperparameter tuning. More broadly, this study underscores the importance of selecting appropriate model architectures and training strategies when tackling complex, non-linear datasets.

5.2. Environmental Impact Assessment

The energy required for training and running the previous models generates a measurable environmental impact that should be carefully assessed.

In an effort to reduce the carbon footprint of our project, we implemented several measures at both the data preprocessing and modelling stages. During preprocessing, we attempted to reduce data dimensionality by removing correlated features. Although this approach did not yield significant improvements in practice, it could have reduced computational costs and energy consumption.

Regarding modelling, we chose k-NN, a training-free method, which inherently avoids the energy-intensive training phase. For our neural network, we employed early stopping to prevent unnecessary epochs and limited the number of random seed trials to three instead of the typical five used in industry benchmarks (Cédric Colas, 2018). In contrast to the Kryptonite-2.0 paper, we avoid using a model with GPT, as running large language models would have added a significant environmental impact. Indeed, LLM training and

inference have been shown to consume millions of litres of freshwater and substantial energy, raising concerns about resource use, water stress in local communities and equity in access (Jegham et al., 2025).

We can calculate the impact of our analysis. We trained the model on computer with a power of 30W during approximately 10 hours, we then consumed 0.3 kWh for the project. For the UK, a reasonable average is 0.12 kg CO_2/kWh (UK Government, 2025) which means that we emitted around 0.04 kg CO_2e which is equivalent to 10-20 chatGPT queries (Sustainability by numbers, 2025). This is much lower than training a single large language model, which can exceed 100,000 kg CO_2e . This project thus demonstrates that meaningful scientific insight does not require energy-intensive computation, contributing positively to discussions around green AI.

In the future, we could adopt more sustainability-oriented research practices. Beyond code optimisation and efficient hyperparameter tuning (such as using Bayesian Search instead of Grid Search) sustainability could be integrated as a constraint in the experimental process itself. For instance, models could be automatically penalized not only for poor accuracy but also for high energy consumption, encouraging the discovery of eco-efficient architectures. Open-sourced libraries also exist, such as CodeCarbon, and could be used to track and better control the Co2 emissions of our project.

5.3. Further work

Future research could expand this comparison to include tree-based methods (e.g., Random Forests, Gradient Boosted Trees) or kernel methods such as SVMs with non-linear kernels. Moreover, ensemble approaches that combine multiple model families could further enhance predictive performance and robustness.

References

- Chattopadhyay, N., Chattopadhyay, A., Gupta, S. S., and Kasper, M. Curse of dimensionality in adversarial examples. In *2019 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2019. doi: 10.1109/IJCNN.2019.8851795.
- Cédric Colas, Olivier Sigaud, P.-Y. O. How many random seeds? statistical power analysis in deep reinforcement learning experiments. Technical report, arXiv, 2018. <https://doi.org/10.48550/arXiv.1806.08295>.
- G. Badaro, M. Saeed, P. P. Transformers for tabular data representation: A survey of models and applications. *Transactions of the Association for Computational Linguistics*, 11, 2023.
- Goodfellow, I., Bengio, Y., and Courville, A. *Deep*

Learning. MIT Press, 2016. <http://www.deeplearningbook.org>.

Jegham, N., Abdelatti, M., Koh, C. Y., Elmoubarki, L., and Hendawi, A. How hungry is ai? benchmarking energy, water, and carbon footprint of llm inference. Technical report, arXiv, 2025. <https://arxiv.org/abs/2505.09598>.

Quinn, H. and Luther, L. Kryptonite-2.0: A simple end to machine learning hype. Technical report, 2025.

Sustainability by numbers. What's the carbon footprint of using chatgpt? <https://www.sustainabilitybynumbers.com/p/carbon-footprint-chatgpt>, 2025. Accessed November 2025.

UK Government. Live uk electricity dashboard. <https://www.energydashboard.co.uk/live>, 2025. Accessed November 2025.

A. Visualisation for the LDA and the PCA

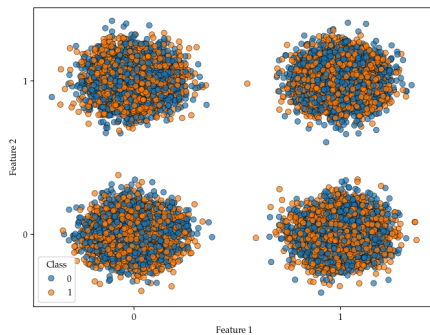


Figure 4. Example Pair of Features Demonstrating Nonlinear Separability

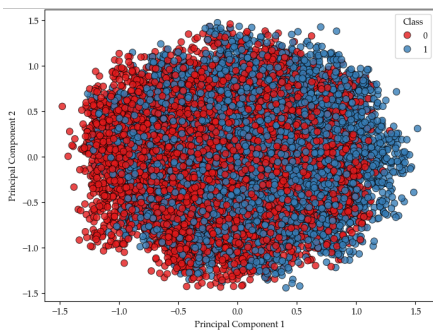


Figure 5. PCA Projection - Not Linearly Separable (PC1: 9.5% 1, PC2: 9.4%)